

Konzeption und Implementierung eines objektorientierten Flugbuchungssystems in Java

Frankfurter Volksbank
Rhein/Main

 **wisag**

MHK
GROUP

 **AKTIVBANK AG**



KULZER
MITSUI CHEMICALS GROUP

Seminararbeit erstellt im Rahmen der Veranstaltung:

Programmieren, Design und Implementierung von Algorithmen II

Studiengruppe:

WI-WS25-II

Name Studierender:

Alexander Fetsch, Nikola Lukic, Ouail Sabraoui,

Enrico Mannino, Neele Sander, Marlon Hüls

Anzahl der Wörter:

2732

(inkl. wörtliche Zitate / Fußnoten)

Anzahl der Wörter:

2700

(exkl. wörtliche Zitate / Fußnoten)

Akademischer Gutachter:

Prof. Dr. Christian Olt

Abgabedatum:

07.06.2026

Inhaltsverzeichnis

ABBILDUNGSVERZEICHNIS.....	I
1 PROBLEMBESCHREIBUNG	1
1.1 PROJEKTNAME UND PROJEKTLOGO	1
1.2 PROBLEMSTELLUNG.....	1
1.3 TEAMÜBERSICHT.....	2
2 TEAMORGANISATION UND TOOLS.....	2
2.1 ZUSAMMENARBEIT IM TEAM.....	2
2.2 EINGESETZTE TOOLS.....	3
3 DARSTELLUNG VON LÖSUNGALTERNATIVEN.....	4
3.1 ARCHITEKTUR: AUFTEILUNG DER GESCHÄFTSLOGIK	4
3.2 DETAILENTSCHEIDUNG: DATENSTRUKTUR DER DATENHALTUNG	5
4 BEGRÜNDETE AUSWAHL EINER LÖSUNG	5
4.1 AUSWAHL DER LOGIKAUFTEILUNG.....	5
4.2 AUSWAHL DER DATENSTRUKTUR	6
5 BESCHREIBUNG DES PROGRAMMS.....	6
5.1 ÜBERSICHT DER HAUPTKOMPONENTEN	6
5.2 BESCHREIBUNG AUSGEWÄHLTER ZENTRALER KLASSEN UND IHRER ROLLEN	6
6 BEISPIELHAFTE PROGRAMMNUTZUNG AUS ANWENDERSICHT	10
7 FAZIT.....	12
8 ANHANG	13
LITERATURVERZEICHNIS	II

Abbildungsverzeichnis

Abbildung 1 Projektlogo	1
Abbildung 2 Ablauf einer Flugbuchung Teil 1	10
Abbildung 3 Ablauf einer Flugbuchung Teil 2	10
Abbildung 4 Ablauf einer Flugbuchung Teil 3	11
Abbildung 5 Ablauf einer Flugbuchung Teil 4	11
Abbildung 6 Ablauf einer Flugbuchung Teil 5	12

1 Problembeschreibung

1.1 Projektname und Projektlogo

Der Name unseres Projektes NOMANE setzt sich aus den Anfangsbuchstaben der Gruppenmitglieder zusammen. NOMANE steht für „Network Oriented Management for Airline Navigation & Efficiency“ und verweist auf das entwickelte Flugbuchungssystem. Das Logo greift diesen Bezug durch die Darstellung eines Flugzeugs sowie einer Flugbahn auf und unterstreicht den Fokus des Projekts auf Luftfahrt, Navigation und Effizienz.



Abbildung 1 Projektlogo

Eigene Erstellung

1.2 Problemstellung

Die Problemstellung besteht darin, ein vereinfachtes Flugbuchungssystem in Form eines objektorientierten Programms zu entwickeln. Im Programm verwalten Fluggesellschaften ihre Flugzeuge und bieten Flüge zwischen Flughäfen an. Passagiere sollen Flüge suchen, buchen, Sitzplätze auswählen sowie Buchungen umbuchen oder stornieren können. Zusätzlich müssen Sitzpläne, Ticketpreise je Kategorie und Gepäckinformationen berücksichtigt werden. Das Programm bildet ausschließlich das Szenario einer Buchung am Schalter vor Ort am Flughafen ab. Für unsere Flüge und Flughäfen wurden bewusst realistische Daten verwendet. Dabei ist jedoch zu beachten, dass im Rahmen des Projektes ausschließlich Direktflüge berücksichtigt werden. Diese Einschränkung ist bewusst gewählt, da Direktverbindungen nicht überall bestehen.

1.3 Teamübersicht

Unser Projektteam besteht aus Neele Sander, Enrico Mannino, Marlon Hüls, Nikola Lukic, Alexander Fetsch und Ouail Sabraoui. Im Folgenden werden nur die Vornamen verwendet.

Die Zusammenarbeit erfolgt arbeitsteilig und in enger Abstimmung. Zu Beginn des Projekts analysierten alle Teammitglieder die Anforderungen und entwickelten die Projektstruktur. Für die Einrichtung der Software investierte jedes Mitglied eine Stunde. Zusätzlich wurden etwa drei Stunden für Planungen und Absprachen aufgewendet.

Alex und Marlon übernahmen die Hauptverantwortung für die Implementierung zentraler Funktionen und investierten hierfür den größten Anteil ihrer Arbeitszeit. Enrico erstellte die Protokolle, sowie das Projektlogo und wendet hierfür fünf Stunden auf. Mit Unterstützung von Ouail erstellte Enrico das UML-Klassendiagramm. Ouail übernahm die Kontrolle und Überprüfung einzelner Inhalte und plante hierfür eine Stunde ein. Nikola bearbeitete den ersten Teil der Ausarbeitung mit etwa 30 Minuten und unterstützte gemeinsam mit Ouail die Sammlung der Flugdaten für 1,5 Stunden. Neele arbeitete an der Ausarbeitung und unterstützte die inhaltliche Überarbeitung, sowie die graphische Gestaltung des UML-Klassendiagramms mit drei Stunden.

In der Testphase beteiligten Neele, Ouail, Nikola und Enrico an der Fehlersuche, sowie der Vereinfachung des Programmcodes mit etwa drei Stunden pro Person. Zusätzlich übernahmen sie gemeinsam die Fertigstellung von Präsentation, Handout und schriftlicher Ausarbeitung, sodass am Ende jeder insgesamt auf etwa 30 Stunden Arbeitsaufwand kommt. Dadurch wird die Arbeit fair verteilt und alle Teammitglieder sind gleichermaßen am Projekt beteiligt.

2 Teamorganisation und Tools

2.1 Zusammenarbeit im Team

Die Zusammenarbeit im Projekt ist arbeitsteilig organisiert. Die Implementierung des Quellcodes übernahmen Marlon und Alex. Enrico übernahm die Protokollführung. Die Vorbereitung der Abschlusspräsentation wurde von Enrico, Ouail, Neele und Nikola getragen, und die schriftliche Ausarbeitung wurde gemeinschaftlich erstellt. Über diese fachliche Aufteilung hinaus folgte das Team einem rotierenden Leitungsmodell. Die Projektleitung wechselte im Projektverlauf zwischen den Teammitgliedern, sodass jede Person koordinative und organisatorische Aufgaben übernahm.

Entscheidungen werden untereinander abgestimmt. Diese erfolgten während der Meetings oder über WhatsApp. Bei einer Pattsituation zählte die Stimme der Projektleitung doppelt.

Meetings fanden in der Regel zweimal wöchentlich statt und dauerten durchschnittlich 150 Minuten. Die Treffen fanden je nach Verfügbarkeit der Teammitglieder in Präsenz, an der Berufsakademie Rhein-Main, oder online über Discord statt. Jedes Meeting wurde protokolliert. Die Protokolle wurden anschließend im GitHub-Repository im Ordner Protokolle abgelegt.

Für die laufende Kommunikation außerhalb der Meetings wurden zwei Plattformen genutzt. WhatsApp diente für kurze Absprachen und spontanen Rückfragen, während über Discord umfangreichere Diskussionen geführt wurden. Der Discord-Server wurde in Kanäle für fachliche Diskussionen, organisatorische Absprachen und Protokolle gegliedert. Ergänzend standen Sprachkanäle für Besprechungen zur Verfügung.

2.2 Eingesetzte Tools

Für die Versionsverwaltung des Quellcodes wurde die Plattform GitHub genutzt. Das Repository wurde zu Projektbeginn von Marlon eingerichtet und steht allen Teammitgliedern zur Verfügung. Neben dem Quellcode wurden auch projektbegleitende Dokumente wie die Sitzungsprotokolle im Repository gespeichert, wodurch eine vollständige Nachvollziehbarkeit aller Änderungen gegeben war.

Um die Funktionalität des Quellcodes sicherzustellen, wurde mit zwei getrennten Entwicklungszweigen gearbeitet. Der Hauptzweig „Main“ enthielt ausschließlich geprüften und lauffähigen Code, während die eigentliche Entwicklung im Zweig „Dev“ erfolgte. Die Implementierung der zentralen Klassen wurde von Marlon und Alex vorgenommen, indem einer der beiden seinen Bildschirm über Discord teilte und sie den Code gemeinsam erarbeiteten. Diese Vorgehensweise entspricht dem Remote Pair Programming (RPP), bei dem die Zusammenarbeit über das Teilen eines Bildschirms (Screen-Sharing) realisiert wird.¹ Auf diese Weise wurden Konflikte durch paralleles Bearbeiten derselben Dateien vermieden. Der im Zweig „Dev“ entstandene Code wurde anschließend in einem Meeting dem Team vorgestellt und gemeinsam getestet. Erst nach erfolgreicher Prüfung wurde der Stand in den Hauptzweig „Main“ übertragen.

¹ Vgl. Schenk et al., 2013, S. 2.

Für das Projektmanagement kam das in GitHub integrierte Kanban Board zum Einsatz. Auf diesem wurden Aufgaben visualisiert, Teammitgliedern zugewiesen und der aktuelle Bearbeitungsstand nachverfolgt. Die Integration in GitHub hatte den Vorteil, dass Code Änderungen und Aufgabenstatus an einer zentralen Stelle einsehbar waren und kein zusätzliches Werkzeug für die Projektsteuerung nötig war.

Der Austausch projektbezogener Dokumente erfolgte je nach Anwendungsfall über die Discord Kanäle für kurzfristige Abstimmungen oder über das GitHub Repository für versionierte und dauerhaft zu archivierende Inhalte. Als einheitliche Entwicklungsumgebung wurde Eclipse verwendet.

3 Darstellung von Lösungsalternativen

Im Verlauf der Programmierung des Flugbuchungssystems standen an einigen Stellen unterschiedliche Lösungswege zur Auswahl. Im Folgenden werden zwei zentrale Entscheidungen dargestellt. Eine betrifft die übergeordnete Architektur des Systems, die andere eine konkrete Detailentscheidung auf der Ebene der Datenhaltung.

3.1 Architektur: Aufteilung der Geschäftslogik

Die folgende Entscheidung betraf die Frage, wie die Geschäftslogik des Programms auf die Klassen verteilt werden sollte. Der erste Entwurf sah eine einzige zentrale Klasse vor, die alle Operationen wie Buchen, Stornieren, Umbuchen, Suchen und die Preisberechnung umfasst. Diese Variante bietet den Vorteil, dass alle Prozesse an einem einzigen Ort zusammengeführt werden und der Einstieg in den Code auf den ersten Blick einfach erscheint. Ein entscheidender Nachteil ist, dass eine solche Klasse mit zunehmendem Funktionsumfang sehr umfangreich, schwer überschaubar und für mehrere unabhängige Verantwortlichkeiten zuständig wird.

Als Alternative wurde eine Aufteilung in mehrere spezialisierte Service-Klassen betrachtet, von denen jede eine fachliche Aufgabe übernimmt. In diesem Ansatz gibt es separate Klassen für das Buchen, Stornieren, Umbuchen, Berechnen von Preisen und Suchen nach Flügen. Obwohl diese Variante zu einer größeren Anzahl von Klassen führt, sorgt sie dafür, dass die einzelnen Klassen kompakt und klar voneinander abgegrenzt bleiben.

3.2 Detailentscheidung: Datenstruktur der Datenhaltung

Auf der Detailebene stellte sich die Frage, welche Datenstruktur für die Speicherung der Objekte in der Datenhaltung verwendet werden sollte. Zur Wahl standen eine einfache Liste sowie eine Schlüssel-Wert-Zuordnung in Form einer HashMap. Eine Liste speichert die Elemente in einer Reihenfolge, in der sie eingefügt wurden. Um ein bestimmtes Element zu finden, müssen jedoch alle gespeicherten Einträge einmal durchlaufen werden. Eine HashMap hingegen weist jedem Objekt einen eindeutigen Schlüssel zu und ermöglicht dadurch einen direkten Zugriff auf ein gesuchtes Element, ohne dass alle anderen Einträge überprüft werden müssen.

Zu Beginn war nur die Liste als Datenstruktur bekannt, weshalb sie für alle Datenmengen verwendet wurde. Im Verlauf der Programmiervorlesung wurde die HashMap und ihre Funktionsweise vorgestellt, wodurch eine zweite, effektivere Möglichkeit für den gezielten Zugriff auf einzelne Objekte entstand.

Die offizielle Java-Lernplattform von Oracle weist darauf hin, dass die Auswahl der passenden Datenstruktur maßgeblich vom beabsichtigten Verwendungszweck abhängt.² Für Datenmengen, die überwiegend anhand eines eindeutigen Bezeichners abgefragt werden, eignet sich eine schlüsselbasierte Struktur, während eine Liste für Mengen, die vor allem vollständig durchlaufen werden, ausreicht.

4 Begründete Auswahl einer Lösung

Für beide dargestellten Entscheidungen wurde jeweils eine der Alternativen ausgewählt. Die Auswahl orientierte sich an der objektorientierten Programmierung sowie an den Anforderungen an Wartbarkeit und Erweiterbarkeit.

4.1 Auswahl der Logikaufteilung

Für die Aufteilung der Geschäftslogik wurde die Variante mit mehreren spezialisierten Service-Klassen gewählt. Das entscheidende Kriterium war das Prinzip der einzigen Verantwortlichkeit (Single-Responsibility-Prinzip), das in der Aufgabenstellung explizit gefordert wird und besagt, dass jede Klasse genau eine zentrale Aufgabe erfüllen soll. Würde man alle Operationen in einer einzigen Klasse zusammenfassen, so wäre dies nicht mit diesem Prinzip vereinbar gewesen. Mit zunehmendem Funktionsumfang wäre sie zu einer schwer wartbaren Sammelklasse

² Vgl. Oracle, 2021, o. S.

geworden. Die gewählte Aufteilung hält die einzelnen Klassen hingegen kompakt und verständlich, und spätere Änderungen werden vereinfacht, da sie auf die jeweils zuständige Klasse beschränkt sind. Der Nachteil der größeren Klassenanzahl wurde angesichts dieser Vorteile bewusst in Kauf genommen.

4.2 Auswahl der Datenstruktur

Bei der Datenstruktur wurde keine einheitliche, sondern eine nach Verwendungszweck differenzierte Entscheidung getroffen. Eine HashMap wurde für die Verwaltung der Buchungen ausgewählt, da Buchungen im laufenden Betrieb regelmäßig anhand ihrer eindeutigen Buchungsnummer abgefragt werden, wie zum Beispiel beim Stornieren oder Umbuchen. Die schlüsselbasierte Struktur ermöglicht hier einen direkten Zugriff, ohne alle Buchungen durchlaufen zu müssen. Die Verwaltung der Flüge und Passagiere erfolgt jedoch mit der ursprünglich verwendeten Liste, da diese Mengen im typischen Ablauf überwiegend vollständig durchlaufen werden, wie bei der Flugsuche oder der Ausgabe aller Flüge, und ein schlüsselbasierter Zugriff hier keinen Mehrwert bietet. Die Auswahl richtete sich somit konsequent nach dem Verwendungszweck der jeweiligen Datenmenge. Die im Verlauf des Projekts neu gewonnene Erkenntnis über die HashMap wurde gezielt dort eingesetzt, wo sie Vorteile bietet, während die Liste an den übrigen Stellen beibehalten wurde.

5 Beschreibung des Programms

5.1 Übersicht der Hauptkomponenten

Das entwickelte Flugbuchungssystem ist in mehrere Klassen und Pakete unterteilt, die unterschiedliche Aufgaben übernehmen. Die model-Klassen bilden dabei die zentralen Objekte des Systems wie Flüge, Passagiere, Sitzplätze und Buchungen ab. Die Service-Klassen stellen die benötigten Funktionen für die Buchungsabwicklung bereit, während die Repository-Klassen die Daten verwalten. Gemeinsam ermöglichen diese Komponenten die Suche nach Flügen, die Durchführung von Buchungen sowie die Verwaltung von Stornierungen und Umbuchungen.

5.2 Beschreibung ausgewählter zentraler Klassen und ihrer Rollen

Das Flugbuchungssystem ist in mehrere Pakete unterteilt. Die Klassen im Paket model repräsentieren die zentralen Bestandteile des Flugbuchungssystems, wie Flüge, Passagiere, Sitzplätze oder Buchungen. Die Klassen im Paket service übernehmen die eigentliche Programmlogik. Die Repositories sind für die Datenhaltung zuständig und die Main-Klasse startet die

KonsolenUI, welche die Konsolenoberfläche bereitstellt. Da das Projekt aus mehreren Klassen besteht, werden im Folgenden nur die für den Buchungsprozess wichtigsten Klassen beschrieben.

Die Klasse Buchung ist das zentrale Verbindungsobjekt des Programms. Sie verknüpft einen Passagier, einen Flug und einen Sitz miteinander. Zusätzlich speichert sie die Buchungsnummer, die Anzahl der Gepäckstücke, den Buchungsstatus und den Gesamtpreis. Beim Erstellen einer Buchung wird automatisch eine eindeutige Buchungsnummer erzeugt und der Status auf „GEBUCHT“ gesetzt. Dadurch bildet diese Klasse den eigentlichen Buchungsvorgang im Programm ab.

Eine der wichtigsten Klassen aus dem Paket model ist die Klasse Flug. Sie repräsentiert einen konkreten Flug mit Flugnummer, Fluggesellschaft, Start- und Zielflughafen, Abflug- und Ankunftszeit, Basispreis sowie dem zugeordneten Flugzeug. Zusätzlich enthält sie Methoden, um die Route als Zeichenkette darzustellen, nach Route und Datum zu prüfen und freie Sitze einer bestimmten Kategorie abzufragen. Dadurch bildet die Klasse Flug die Grundlage für die Suche, Auswahl und Buchung eines Fluges.

Die Klasse Flugzeug beschreibt das eingesetzte Flugzeug und verwaltet dessen Sitzplätze. Hierzu enthält sie eine Liste von Sitzobjekten, die den verschiedenen Sitzkategorien First, Business und Economy zugeordnet sind. Beim Erstellen eines Flugzeugs werden diese Sitze automatisch erzeugt und der entsprechenden Kategorie zugewiesen. Außerdem kann die Klasse freie Sitze einer bestimmten Kategorie zurückgeben oder einen Sitz anhand seiner Sitznummer suchen. Damit ist Flugzeug wichtig für die Sitzplatzverwaltung und die Prüfung, ob ein gewünschter Sitz existiert.

Eng damit verbunden ist die Klasse Sitz. Sie enthält die Sitzreihe, den Platz, die Sitzkategorie und den Belegungsstatus. Durch die Methoden belegen() und freigeben() kann der Status eines Sitzplatzes verändert werden. Diese Klasse ist deshalb wichtig, weil sie die konkrete Verfügbarkeit eines Sitzplatzes während Buchung, Stornierung und Umbuchung abbildet.

Die Klasse Passagier speichert die Kundendaten wie Vorname, Nachname, E-Mail-Adresse und Telefonnummer. Sie ist bewusst einfach gehalten und dient hauptsächlich als Datenobjekt. Über sie kann eine Buchung eindeutig einem Kunden zugeordnet werden.

Der BuchungService im Paket „service“ übernimmt die zentrale Logik für die Durchführung von Flugbuchungen. Beim Buchen prüft der Service zunächst, ob die eingegebene Sitznummer gültig ist und ob der gewünschte Sitz noch verfügbar ist. Anschließend wird mithilfe des PreisService der Gesamtpreis berechnet, der Sitz belegt und die neue Buchung im BuchungsRepository abgelegt. Zusätzlich leitet der BuchungService Stornierungen an den StornoService weiter. Dadurch koordiniert die Klasse den Ablauf zwischen Buchung, Preisberechnung, Sitzverwaltung und Stornierung.

Der PreisService kapselt die Preislogik des Programms. Er berechnet den Gesamtpreis aus dem Basispreis des Fluges, dem Multiplikator der gewählten Sitzkategorie und einer möglichen Gepäckgebühr. Dadurch ist die Preisberechnung nicht direkt in der Buchungsklasse enthalten, sondern in einer eigenen Serviceklasse ausgelagert. Dies entspricht dem Prinzip der Trennung von Verantwortlichkeiten.

Für die Suche nach passenden Flügen wird der FlugSuchService verwendet. Er greift auf das Flug-Repository zu und filtert die vorhandenen Flüge nach Startflughafen, Zielflughafen und optional nach Datum. Dadurch stellt die Klasse die zentrale Suchfunktion des Programms bereit und ermöglicht eine gezielte Auswahl geeigneter Flüge.

Ergänzend übernehmen der StornoService und der UmbuchungsService spezielle Abläufe nach einer bestehenden Buchung. Der StornoService berechnet abhängig vom Zeitraum bis zum Abflug eine Stornogebühr, gibt den Sitz wieder frei und setzt den Status auf „STORNIERT“. Der UmbuchungsService prüft bei einer Umbuchung den neuen Sitz, gibt den alten Sitz frei, belegt den neuen Sitz und aktualisiert Flug, Sitz, Status und Preis der Buchung.

Die Repository-Klassen dienen der Datenhaltung. Beispielsweise speichert das BuchungsRepository Buchungen in einer HashMap, wodurch Buchungen anhand ihrer Buchungsnummer effizient gefunden werden können.

Die Klasse Datenerhaltung im Paket „service“ dient der dauerhaften Speicherung der Anwendungsdaten. Die Klasse fasst alle zentralen Repositories der Flughäfen, Flüge, Buchungen und Passagiere in einem serialisierbaren Objekt zusammen. Dadurch können die aktuellen Daten beim Beenden des Programms gespeichert und beim nächsten Start wieder geladen werden. Sie unterstützt somit die Persistenz der Daten und ermöglicht die Wiederverwendung gespeicherter Daten über mehrere Programmausführungen hinweg.

Zusätzlich enthält das Programm ein Paket mit Interfaces. Diese Interfaces legen fest, welche Methoden von den jeweiligen Service- und Repository-Klassen angeboten werden müssen. Dies entspricht dem von Ullenboom beschriebenen Konzept von Schnittstellen, bei dem die geforderten Funktionen definiert werden, während die konkrete Implementierung in den jeweiligen Klassen erfolgt.³

Ergänzend enthält das Programm ein eigenes Paket „exception“ für die Fehlerbehandlung. Die darin enthaltenen Exception-Klassen werden verwendet, um typische Fehlerfälle wie nicht gefundene Buchungen, nicht verfügbare Flüge, bereits belegte Sitzplätze oder ungültige Sitznummern abzufangen. Dadurch können Fehler eindeutig identifiziert und verständlich ausgegeben werden.

Die Klasse Main bildet den Einstiegspunkt des Flugbuchungssystems. Die Klasse initialisiert die benötigten Repositories und Services und lädt beim Programmstart vorhandene Anwendungsdaten. Sind keine gespeicherten Daten vorhanden, werden die Repositories neu erstellt und mit Testdaten befüllt. Anschließend erstellt und verknüpft sie die benötigten Services und startet die Klasse KonsolenUI.

Die Klasse KonsolenUI stellt dabei die Benutzerschnittstelle des Flugbuchungssystems dar. Die Klasse steuert das Konsolenmenü, verarbeitet Nutzereingaben und gibt Ergebnisse auf der Konsole aus. Sie ermöglicht unter anderem die Flugsuche, Flugbuchungen, Stornierungen, Umbuchungen sowie die Anzeige von Sitzplänen und Verwaltungsinformationen. Die eigentlichen Aufgaben des Programms werden von den Service- und Repository-Klassen übernommen. Die KonsolenUI ist lediglich für die Eingabe und Ausgabe zuständig und dient als Verbindung zwischen dem Nutzer und den anderen Programmkomponenten.

Das Zusammenspiel der beschriebenen Klassen ermöglicht die Umsetzung des Buchungsprozesses. Ein Passagier kann einen Flug auswählen und einen verfügbaren Sitzplatz buchen. Die einzelnen Komponenten des Programms arbeiten dabei zusammen, um Buchungen zu verwalten und Änderungen wie Stornierungen oder Umbuchungen zuverlässig umzusetzen.

³ Vgl. Ullenboom, 2025, Kapitel 8.1

6 Beispielhafte Programmnutzung aus Anwendersicht

Nach dem Programmstart lädt das System die Testdaten und zeigt ein Hauptmenü mit zehn Optionen an. Der Anwender wählt Flug buchen und gibt seine E-Mail-Adresse ein. Da es sich um einen neuen Passagier handelt, werden zusätzlich Namen und Telefonnummer abgefragt. Anschließend werden alle verfügbaren Flüge aufgelistet, woraus der Nutzer Flug LH400 auswählt (Frankfurt → New York). Nach Anzeige des Sitzplans wählt der Anwender Sitz D1 in der Business Class und gibt zwei Gepäckstücke an. Das System bestätigt die Buchung BU-0001 mit 779,99€ und kehrt ins Hauptmenü zurück.

```
Testdaten geladen: 6 Flughäfen | 3 Airlines | 5 Flugzeuge | 12 Flüge | 5 Passagiere
```

```
=====
FLUGBUCHUNGSSYSTEM - Menü
=====
1 - Flüge suchen
2 - Flug buchen
3 - Buchung stornieren
4 - Buchung umbuchen
5 - Sitzplan anzeigen
6 - Freie Sitze anzeigen
7 - Alle Buchungen anzeigen
8 - Auslastung aller Flüge
9 - Gepäckübersicht für einen Flug
0 - Beenden
-----
Deine Wahl:
```

Abbildung 2 Ablauf einer Flugbuchung Teil 1

Eigene Erstellung

```
Testdaten geladen: 6 Flughäfen | 3 Airlines | 5 Flugzeuge | 12 Flüge | 5 Passagiere
```

```
=====
FLUGBUCHUNGSSYSTEM - Menü
=====
1 - Flüge suchen
2 - Flug buchen
3 - Buchung stornieren
4 - Buchung umbuchen
5 - Sitzplan anzeigen
6 - Freie Sitze anzeigen
7 - Alle Buchungen anzeigen
8 - Auslastung aller Flüge
9 - Gepäckübersicht für einen Flug
0 - Beenden
-----
Deine Wahl: 2
--- Flug buchen ---
E-Mail des Passagiers:
```

Abbildung 3 Ablauf einer Flugbuchung Teil 2

Eigene Erstellung

Testdaten geladen: 6 Flughäfen | 3 Airlines | 5 Flugzeuge | 12 Flüge | 5 Passagiere

```
=====
FLUGBUCHUNGSSYSTEM - Menü
=====
1 - Flüge suchen
2 - Flug buchen
3 - Buchung stornieren
4 - Buchung umbuchen
5 - Sitzplan anzeigen
6 - Freie Sitze anzeigen
7 - Alle Buchungen anzeigen
8 - Auslastung aller Flüge
9 - Gepäckübersicht für einen Flug
0 - Beenden
=====
Deine Wahl: 2

--- Flug buchen ---
E-Mail des Passagiers: maxmustermann@test.de
Neuer Passagier - bitte Daten eingeben:
Vorname: Max
Nachname: Mustermann
Telefon: 017777777
Passagier angelegt: Max Mustermann

Verfügbare Flüge:
LH400 | FRA - JFK | 2026-06-01T10:00 -> 2026-06-01T20:30
LH402 | FRA - JFK | 2026-07-05T11:00 -> 2026-07-05T21:30
LH200 | MUC - FRA | 2026-06-01T08:00 -> 2026-06-01T09:15
LH202 | MUC - FRA | 2026-07-03T07:30 -> 2026-07-03T08:45
LH300 | FRA - LHR | 2026-06-10T09:00 -> 2026-06-10T10:45
LH302 | FRA - LHR | 2026-07-14T14:00 -> 2026-07-14T15:45
EK051 | DXB - FRA | 2026-06-01T10:00 -> 2026-06-01T17:10
EK052 | DXB - FRA | 2026-06-15T15:00 -> 2026-06-15T22:10
AF100 | FRA - CDG | 2026-06-05T07:00 -> 2026-06-05T08:30
AF102 | FRA - CDG | 2026-07-20T16:00 -> 2026-07-20T17:30
AF200 | CDG - JFK | 2026-07-01T10:30 -> 2026-07-01T19:00
AF202 | CDG - JFK | 2026-08-10T11:00 -> 2026-08-10T19:30
Flugnummer:
```

Abbildung 4 Ablauf einer Flugbuchung Teil 3

Eigene Erstellung

```
Verfügbare Flüge:
LH400 | FRA - JFK | 2026-06-01T10:00 -> 2026-06-01T20:30
LH402 | FRA - JFK | 2026-07-05T11:00 -> 2026-07-05T21:30
LH200 | MUC - FRA | 2026-06-01T08:00 -> 2026-06-01T09:15
LH202 | MUC - FRA | 2026-07-03T07:30 -> 2026-07-03T08:45
LH300 | FRA - LHR | 2026-06-10T09:00 -> 2026-06-10T10:45
LH302 | FRA - LHR | 2026-07-14T14:00 -> 2026-07-14T15:45
EK051 | DXB - FRA | 2026-06-01T10:00 -> 2026-06-01T17:10
EK052 | DXB - FRA | 2026-06-15T15:00 -> 2026-06-15T22:10
AF100 | FRA - CDG | 2026-06-05T07:00 -> 2026-06-05T08:30
AF102 | FRA - CDG | 2026-07-20T16:00 -> 2026-07-20T17:30
AF200 | CDG - JFK | 2026-07-01T10:30 -> 2026-07-01T19:00
AF202 | CDG - JFK | 2026-08-10T11:00 -> 2026-08-10T19:30
Flugnummer: LH400
--- Sitzplan: LH400 (FRA - JFK) ---
A1 | FIRST | FREI
A2 | FIRST | FREI
B1 | FIRST | FREI
B2 | FIRST | FREI
C1 | BUSINESS | FREI
C2 | BUSINESS | FREI
C3 | BUSINESS | FREI
C4 | BUSINESS | FREI
C5 | BUSINESS | FREI
D1 | BUSINESS | FREI
D2 | BUSINESS | FREI
D3 | BUSINESS | FREI
D4 | BUSINESS | FREI
D5 | BUSINESS | FREI
E1 | BUSINESS | FREI
E2 | BUSINESS | FREI
E3 | BUSINESS | FREI
E4 | BUSINESS | FREI
E5 | BUSINESS | FREI
F1 | BUSINESS | FREI
F2 | BUSINESS | FREI
F3 | BUSINESS | FREI
F4 | BUSINESS | FREI
F5 | BUSINESS | FREI
G1 | ECONOMY | FREI
G2 | ECONOMY | FREI
G3 | ECONOMY | FREI
G4 | ECONOMY | FREI
G5 | ECONOMY | FREI
G6 | ECONOMY | FREI
H1 | ECONOMY | FREI
H2 | ECONOMY | FREI
H3 | ECONOMY | FREI
H4 | ECONOMY | FREI
H5 | ECONOMY | FREI
H6 | ECONOMY | FREI
I1 | ECONOMY | FREI
I2 | ECONOMY | FREI
I3 | ECONOMY | FREI
I4 | ECONOMY | FREI
```

Abbildung 5 Ablauf einer Flugbuchung Teil 4

Eigene Erstellung

```

U3 | ECONOMY | FREI
U4 | ECONOMY | FREI
U5 | ECONOMY | FREI
U6 | ECONOMY | FREI
V1 | ECONOMY | FREI
V2 | ECONOMY | FREI
V3 | ECONOMY | FREI
V4 | ECONOMY | FREI
V5 | ECONOMY | FREI
V6 | ECONOMY | FREI
M1 | ECONOMY | FREI
M2 | ECONOMY | FREI
M3 | ECONOMY | FREI
M4 | ECONOMY | FREI
M5 | ECONOMY | FREI
M6 | ECONOMY | FREI
X1 | ECONOMY | FREI
X2 | ECONOMY | FREI
X3 | ECONOMY | FREI
X4 | ECONOMY | FREI
X5 | ECONOMY | FREI
M6 | ECONOMY | FREI
V1 | ECONOMY | FREI
V2 | ECONOMY | FREI
V3 | ECONOMY | FREI
V4 | ECONOMY | FREI
V5 | ECONOMY | FREI
V6 | ECONOMY | FREI
Z1 | ECONOMY | FREI
Z2 | ECONOMY | FREI
Z3 | ECONOMY | FREI
Z4 | ECONOMY | FREI
Z5 | ECONOMY | FREI
D6 | ECONOMY | FREI
Sitznummer (z.B. A1): D1
Anzahl Gepäckstücke (mind. 1): 2
Buchung erfolgreich!
BU-0001 | Max Mustermann | FRA - JFK | D1 | GEBUCHT | 779,99 EUR
=====
FLUGBUCHUNGSSYSTEM - Menu
=====
1 - Flüge suchen
2 - Flug buchen
3 - Buchung stornieren
4 - Buchung umbuchen
5 - Sitzplan anzeigen
6 - Freie Sitze anzeigen
7 - Alle Buchungen anzeigen
8 - Auslastung aller Flüge
9 - Gepäckübersicht für einen Flug
0 - Beenden
=====
Definie Wahl:
  
```

Abbildung 6 Ablauf einer Flugbuchung Teil 5

Eigene Erstellung

7 Fazit

Das entwickelte Flugbuchungssystem bildet die zentralen Abläufe einer Flugbuchung in vereinfachter, aber praxisnaher Form ab. Durch den modularen und objektorientierten Aufbau konnten die wichtigsten Funktionen wie Flugsuche, Buchung, Umbuchung, Stornierung, Sitzplatzverwaltung und Gepäckübersicht strukturiert umgesetzt werden. Die Arbeit an dem Projekt hat gezeigt, wie sich fachliche Anforderungen in eine gegliederte Architektur überführen lassen. Gleichzeitig wurde deutlich, dass die bewussten Vereinfachungen, insbesondere die Beschränkung auf Direktflüge und die Konzentration auf eine Buchung am Schalter, die Implementierung übersichtlicher machen und eine gute Grundlage für mögliche Erweiterungen schaffen. Zukünftige Erweiterungen könnten beispielsweise die Unterstützung von Umstiegen, eine grafische Benutzeroberfläche oder die automatisierte regelmäßige Erstellung der Flüge umfassen.

Literaturverzeichnis

1. Schenk, J., Prechelt, L. & Salinger, S. (2013): Distributed-Pair Programming can work well and is not just Distributed Pair-Programming. Abgerufen am 26.05.2026 von <https://arxiv.org/pdf/1311.6249>
2. Oracle (2021): Storing Data Using the Collections Framework. Abgerufen am 29.05.2026 von <https://dev.java/learn/api/collections-framework/intro/>
3. Ullenboom, C. (2025): Java ist auch eine Insel (18.Auflage). Rheinwerk Verlag. Abgerufen am 06.05.2026 von https://openbook.rheinwerk-verlag.de/javainsel/08_001.html-u8
4. Adobe Firefly: Verwendung zur Logo Erstellung. Abgerufen am 06.05.2026 von <https://firefly.adobe.com/generate/image>
5. ChatGPT: Verbesserung der Rechtschreibung. Abgerufen am 06.05.2026 von <https://chatgpt.com/c/69f07651-32ec-83eb-a3b3-053ee5491e38>
6. Claude: Fehlersuche und Ideenfindung im Code. Abgerufen am 06.05.2026 von <https://claude.ai/>

Eidesstattliche Erklärung

Hiermit erklären wir, dass wir die vorliegende Ausarbeitung selbständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt und die aus fremden Quellen direkt oder indirekt übernommenen Gedanken als solche kenntlich gemacht haben. Die Arbeit oder Teile hieraus wurde und wird keiner anderen Stelle oder anderen Person im Rahmen einer Prüfung vorgelegt. Wir versichern zudem, dass keine sachliche Übereinstimmung mit einer im Rahmen eines vorangegangenen Studiums angefertigten Seminar-, Haus-, Diplom- oder Abschlussarbeit sowie Bachelor-Thesis besteht.

Enrico Mannino, 251155



Ouail Sabraoui, 251106



Alexander Fetsch, 251032



Nikola Lukic, 251126



Marlon Hüls, 251023



Neele Sander, 251110

